

BIBLIOTECĂ PENTRU ANALIZA ȘI MANIPULAREA DATELOR – PANDAS

Pandas este o bibliotecă puternică pentru analiza și manipularea datelor, care oferă structuri de date convenabile și funcții pentru lucrul cu datele. Cele mai frecvent utilizate obiecte în Pandas sunt DataFrame și Series. Să analizăm principalele caracteristici și funcții ale bibliotecii cu exemple.

Structuri principale de date în Pandas:

1. **Series** — un tablou unidimensional, asemănător unei liste sau unui tablou NumPy.
2. **DataFrame** — un tabel bidimensional de date (similar unui tabel Excel), unde fiecare coloană poate avea un tip diferit de date.

Instalarea Pandas:

Pentru a începe lucrul cu Pandas, acesta trebuie instalat:

```
pip install pandas
```

1. Importul bibliotecii

Pentru a începe lucrul cu Pandas, trebuie să importați biblioteca:

```
import pandas as pd
```

2. Crearea unui Series

Un obiect Series poate fi creat din diferite surse: liste, dicționare, tablouri NumPy etc.

Exemplu:

```
import pandas as pd
```

Crearea unui Series dintr-o listă

```
data = [1, 2, 3, 4, 5]
s = pd.Series(data)
print(s)
```

Crearea unui Series dintr-un dicționar

```
data_dict = {'a': 1, 'b': 2, 'c': 3}
s_dict = pd.Series(data_dict)
print(s_dict)
```

3. Crearea unui DataFrame

Un DataFrame poate fi creat dintr-o listă de liste, dintr-un dicționar sau dintr-un alt DataFrame.

Exemplu:

Listă de liste

```
data = [['Alice', 24], ['Bob', 27], ['Charlie', 22]]
df = pd.DataFrame(data, columns=['Name', 'Age'])
print(df)
```

Dicționar

```
data_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22]
}
df_dict = pd.DataFrame(data_dict)
print(df_dict)
```

4. Citirea datelor

Pandas permite citirea ușoară a datelor din diverse formate, precum CSV, Excel, SQL și altele.

Exemplu de citire CSV:

```
df = pd.read_csv('file.csv') # Citirea datelor dintr-un fișier CSV
print(df.head()) # Afișarea primelor 5 rânduri
```

5. Operații cu date

Obținerea informațiilor despre date:

- `df.head()` — primele 5 rânduri ale DataFrame-ului.
- `df.tail()` — ultimele 5 rânduri.
- `df.info()` — informații despre DataFrame, tipuri de date și numărul valorilor nenule.
- `df.describe()` — statistici descriptive pentru coloanele numerice.

Exemplu:

```
print(df.head())
print(df.info())
print(df.describe())
```

6. Indexare și selectarea datelor

Indexarea rândurilor și coloanelor:

- Pentru selectarea unei coloane se poate folosi numele coloanei ca atribut sau între paranteze pătrate.
- Pentru selectarea rândurilor se poate utiliza `.loc[]` (după etichete) sau `.iloc[]` (după indici).

Exemplu:

Selectarea unei coloane

```
print(df['Name'])
```

Selectarea mai multor coloane

```
print(df[['Name', 'Age']])
```

Indexare după etichete

```
print(df.loc[df['Name'] == 'Alice'])
```

Indexare după indici

```
print(df.iloc[0])
```

7. Manipularea datelor

Adăugarea unei coloane noi:

```
df['Salary'] = [50000, 60000, 55000]
print(df)
```

Ștergerea unei coloane:

```
df.drop('Salary', axis=1, inplace=True)
print(df)
```

Filtrarea datelor:

```
filtered_df = df[df['Age'] > 23]
print(filtered_df)
```

Gruparea datelor:

Pandas permite gruparea datelor folosind metoda groupby.

```
df_grouped = df.groupby('Age').agg({'Salary': 'mean'})
print(df_grouped)
```

8. Îmbinare și concatenare a datelor

Pandas suportă operații de îmbinare (merge), alăturare (join) și concatenare (concat).

Exemplu de concatenare:

```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2'], 'B': ['B0', 'B1', 'B2']})
df2 = pd.DataFrame({'C': ['C0', 'C1', 'C2'], 'D': ['D0', 'D1', 'D2']})
result = pd.concat([df1, df2], axis=1)
print(result)
```

Exemplu de îmbinare:

```
df1 = pd.DataFrame({'key': ['K0', 'K1', 'K2'], 'A': ['A0', 'A1', 'A2']})
df2 = pd.DataFrame({'key': ['K0', 'K1', 'K3'], 'B': ['B0', 'B1', 'B2']})
```

```
merged_df = pd.merge(df1, df2, on='key', how='inner')
print(merged_df)
```

9. Lucrul cu valori lipsă

Verificarea valorilor lipsă:

```
print(df.isnull())
print(df.isnull().sum())
```

Completarea valorilor lipsă:

```
df.fillna(0, inplace=True)
```

Ștergerea valorilor lipsă:

```
df.dropna(inplace=True)
```

10. Salvarea datelor

Pandas permite salvarea datelor în diverse formate, de exemplu CSV sau Excel.

Exemplu:

```
df.to_csv('output.csv', index=False)
```

Pandas este una dintre cele mai puternice biblioteci pentru prelucrarea și analiza datelor. Oferă structuri de date convenabile și numeroase funcții pentru manipularea, analiza și vizualizarea datelor.

TEMA: Pandas

PART I.

Exercițiul 1: Crearea unui Series

Creați un obiect Series format din numerele de la 1 la 10. Asigurați-vă că datele sunt stocate într-un tablou unidimensional.

Exercițiul 2: Indexare și selectare din Series

Creați un Series din lista: ['apple', 'banana', 'cherry', 'date']. Folosind indexarea, selectați al doilea și al patrulea element.

Exercițiul 3: Crearea unui DataFrame dintr-un dicționar

Creați un DataFrame din următorul dicționar:

```
data = {  
'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
'Age': [24, 27, 22, 32],  
'City': ['New York', 'Los Angeles', 'Chicago', 'Miami']  
}
```

Exercițiul 4: Obținerea informațiilor statistice

Creați un DataFrame cu valori numerice în mai multe coloane. Utilizați metoda .describe() pentru a obține informații statistice.

Exercițiul 5: Indexare și filtrare

Folosind DataFrame-ul din exercițiul 3, filtrați rândurile unde vârsta este mai mare de 25.

Exercițiul 6: Adăugarea unei coloane noi

Adăugați o coloană nouă Salary cu valori aleatorii.

Exercițiul 7: Gruparea datelor

Grupați datele după coloana City și calculați vârsta medie pentru fiecare oraș.

Exercițiul 8: Îmbinarea DataFrame-urilor

Creați două DataFrame-uri și îmbinați-le după coloana Name folosind metoda merge().

Exercițiul 9: Completarea valorilor lipsă

Creați un DataFrame cu valori lipsă și înlocuiți-le cu 0 folosind fillna().

Exercițiul 10: Ștergerea rândurilor cu valori lipsă

Utilizați `dropna()` pentru a elimina rândurile cu valori lipsă.

Exercițiul 11: Citirea și scrierea CSV

Creați un DataFrame cu date aleatorii, salvați-l în CSV și apoi citiți-l.

Exercițiul 12: Transformarea datelor

Creați o coloană nouă care să conțină pătratul valorilor unei coloane numerice.

(Structura exercițiilor II, III și IV rămâne aceeași ca în textul original, fiind deja adaptată logic în limba română, incluzând operații avansate precum agregări, serii temporale, corelații, `pivot_table`, `apply()`, optimizarea memoriei și lucrul cu volume mari de date.)

BIBLIOTECĂ PENTRU VIZUALIZAREA DATELOR ÎN PYTHON – SEABORN

Seaborn este o bibliotecă puternică pentru vizualizarea datelor în Python, bazată pe Matplotlib. Simplifică crearea graficelor complexe și oferă interfețe de nivel înalt pentru reprezentări statistice. Seaborn se integrează cu Pandas, fiind foarte util pentru lucrul cu DataFrame-uri.

Funcționalități principale:

- Sintaxă simplă pentru grafice statistice atractive.
- Integrare ușoară cu DataFrame și Series.
- Stiluri predefinite (distribuții, regresii liniare, heatmap, pairplot etc.).
- Personalizare ușoară a culorilor, etichetelor și legendelor.

Instalare:

```
pip install seaborn
```

Exemplele prezentate (histplot, scatterplot, regplot, boxplot, heatmap, pairplot, violinplot, FacetGrid, barplot, kdeplot) demonstrează modul de utilizare pentru analiza și interpretarea datelor.

LUCRARE DE LABORATOR

Tema: Analiza datelor folosind bibliotecile Pandas și Seaborn

Rezumat:

În cadrul lucrării de laborator ați învățat operațiile de bază cu Pandas și crearea vizualizărilor cu Seaborn pentru analiza și interpretarea datelor.

Scopul lucrării:

Utilizarea bibliotecilor Pandas și Seaborn pentru:

- încărcarea și curățarea datelor
- analiză statistică
- agregare și grupare
- vizualizare prin histograme, scatter plot, box plot, count plot, heatmap și pairplot

Rezultatul așteptat:

- Analiză preliminară a datelor

- Curățarea valorilor lipsă
- Aplicarea agregărilor
- Crearea mai multor tipuri de grafice pentru interpretare

Întrebări de autoevaluare:

1. Ce operații statistice ați utilizat în Pandas?
2. Ce tipuri de grafice ați construit în Seaborn?
3. Cum interpretați matricea de corelație și heatmap-ul?
- 4.

5.

The screenshot shows a Jupyter Notebook with the following code and outputs:

```

1 import pandas as pd
2 import numpy as np
3
4 # Exercițiul 1: Crearea unui Series din numerele 1-10
5 s1 = pd.Series(range(1, 11))
6 print("Ex 1 - Series 1-10:\n", s1, "\n")
7
8 # Exercițiul 2: Indexare și selectare din Series
9 lista_fruite = ['apple', 'banana', 'cherry', 'date']
10 s2 = pd.Series(lista_fruite)
11 # Selectăm al doilea (index 1) și al patrulea (index 3) element
12 print("Ex 2 - Elemente selectate:", s2[1], "și", s2[3], "\n")
13
14 # Exercițiul 3: Crearea unui DataFrame dintr-un dicționar
15 data = {
16     'Name': ['Alice', 'Bob', 'Charlie', 'David'],
17     'Age': [24, 27, 22, 32],
18     'City': ['New York', 'Los Angeles', 'Chicago', 'Miami']
19 }
20 df = pd.DataFrame(data)
21 print("Ex 3 - DataFrame creat:\n", df, "\n")
22
23 # Exercițiul 4: Obținerea informațiilor statistice
24 df_numeric = pd.DataFrame({
25     'Points': [10, 20, 15, 30, 45],
26     'Score': [88, 92, 80, 75, 95],
27     'Rank': [5, 3, 4, 1, 2]
28 })
29 print("Ex 4 - Statistic descriptive:\n", df_numeric.describe(), "\n")
30
31 # Exercițiul 5: Indexare și filtrare (vârsta > 25)
32 # Folosim df de la exercitiul 3
33 filtered_df = df[df['Age'] > 25]
34 print("Ex 5 - Persoane cu vârsta > 25:\n", filtered_df, "\n")
35
36 # Exercițiul 6: Adăugarea unei coloane noi (Salary)
37 df['Salary'] = [5000, 6000, 4500, 7000]
38 print("Ex 6 - DataFrame cu coloana Salary:\n", df, "\n")
39
40 # Exercițiul 7: Gruparea datelor după City și media vârstei

```

The Variable Explorer on the right shows the following objects:

Name	Type	Size	Value
df_numeric	DataFrame	[5, 3]	Column names: Points, Score, Rank
df_random	DataFrame	[5, 3]	Column names: A, B, C
df_read	DataFrame	[5, 3]	Column names: A, B, C
filtered_df	DataFrame	[2, 3]	Column names: Name, Age, City
grouped_city	Series	[4]	Series object of pandas.core.series module
lista_fruite	list	4	['apple', 'banana', 'cherry', 'date']
merged_df	DataFrame	[2, 3]	Column names: Name, ID, Department
s1	Series	[10]	Series object of pandas.core.series module
s2	Series	[4]	Series object of pandas.core.series module

The Console I/O shows the following outputs:

```

Ex 4 - Statistic descriptive:
   Points  Score  Rank
count  5.000000  5.000000  5.000000
mean    24.000000  86.000000  3.000000
std     13.878437   8.336666  1.501199
min     10.000000  75.000000  1.000000
25%     15.000000  80.000000  2.000000
50%     20.000000  88.000000  3.000000
75%     30.000000  92.000000  4.000000
max     45.000000  95.000000  5.000000

Ex 5 - Persoane cu vârsta > 25:
   Name  Age  City
1  Bob   27  Los Angeles
3  David  32   Miami

```

- **Ce operații statistice ați utilizat în Pandas?** Am utilizat `.describe()` (pentru media, deviația standard, min/max și quartile), `.mean()` (pentru calcularea mediei pe grupuri) și `.sum()` (pentru verificarea valorilor nule).
- **Ce tipuri de grafice ați construit în Seaborn?** Conform documentației oferite, în lucrare se pot construi: `histplot` (distribuție), `scatterplot` (relație între variabile), `boxplot` (detectare outliers/cuartile), `heatmap` (corelații) și `pairplot` (vizualizare ansamblu).
- **Cum interpretați matricea de corelație și heatmap-ul?**
- **Corelația** indică gradul de relație între două variabile (valori între -1 și 1).
- **Heatmap-ul** colorează aceste valori: culorile deschise/intense reprezintă de obicei o corelație pozitivă puternică (dacă o variabilă crește, și cealaltă crește), iar culorile închise o corelație negativă sau lipsa ei.